LAB 01 Professional setup, Git & GitHub contribution workflow

Unit 1 • 3-hour guided laboratory • CLO-5

Learning goal. Install the complete Android development toolchain, deploy an app to a REAL Android phone, authenticate GitHub with an SSH key, and contribute through reviewed pull requests.

Core concepts. Git working tree / staging / commits / branches / remotes; ED25519 authentication; Android SDK Manager; Flutter SDK, platform tools and ADB; collaboration and reviews.

Before you start. A Windows 10/11 laptop is the illustrated host. Linux/macOS commands are noted where different. Have a GitHub account, internet connection, USB data cable and a physical Android phone.

A. Install Git, Android Studio and the complete Android toolchain

STEP 01 | Install Git and Git Bash

Download Git for Windows from https://git-scm.com/downloads. Install Git Bash with OpenSSH available, then verify the two programs in Git Bash. Use your normal account instead of running routine Git work as administrator.

STEP 02 | Install Android Studio

Download the current stable Android Studio from https://developer.android.com/studio. Complete the Setup Wizard, choose the standard Android SDK setup, accept the licenses and allow the downloads to finish.

STEP 03 | Download an Android SDK platform

Open Android Studio Welcome screen > More Actions > SDK Manager (or Tools > SDK Manager from an open project). On the SDK Platforms tab, select a supported stable Android API platform and click Apply.

STEP 04 | Install the required SDK Tools

In SDK Manager > SDK Tools, select the required tools listed in the table below, then click Apply. The emulator is optional because these labs use a physical Android phone. NDK (Side by side) and CMake are only needed if a plugin or build explicitly requests them.

Component	Where to select	Required for phone?
Android SDK Platform	SDK Platforms tab	Yes
Android SDK Platform-Tools (adb)	SDK Tools tab	Yes
Android SDK Build-Tools	SDK Tools tab	Yes
Android SDK Command-line Tools (latest)	SDK Tools tab	Yes
Android Emulator	SDK Tools tab	No - optional
Google/OEM USB Driver (Windows)	SDK Tools / manufacturer	Only if needed
NDK (Side by side), CMake	SDK Tools tab	Only if project needs them

STEP 05 | Locate the Android SDK and enable Flutter plugins

Copy the Android SDK Location shown in SDK Manager; do not guess the installed path. On Windows, a common location is shown below. In Android Studio, open Settings > Plugins > Marketplace, install Flutter, accept the Dart plugin installation and restart the IDE.

```
%LOCALAPPDATA%\Android\Sdk    (common default; verify in SDK Manager)
```

Plugin walkthrough: Flutter and Dart in Android Studio

PLUGIN 1 | Open the Marketplace

Windows/Linux: File > Settings (Ctrl+Alt+S) > Plugins > Marketplace. macOS: Android Studio > Settings > Plugins. Ensure the Marketplace tab, not Installed, is selected.

PLUGIN 2 | Install the Flutter plugin

Search for Flutter, select the plugin published for Flutter development, and click Install. Approve any permission or compatibility dialog shown by your installed IDE version.

PLUGIN 3 | Install the Dart plugin

When Android Studio asks to install or enable the Dart plugin as a dependency of Flutter, select Install/Enable. If no prompt appears, open Plugins > Installed and confirm BOTH Flutter and Dart are enabled. If Dart is missing, search for Dart in Marketplace and install it.

PLUGIN 4 | Restart and locate Flutter settings

Restart Android Studio. Verify Flutter and Dart appear enabled in Plugins > Installed. The New Flutter Project option should now appear. Some IDE versions also expose Languages & Frameworks > Flutter to set the Flutter SDK location.

PLUGIN 5 | Distinguish the plugin from the SDK

The Android Studio plugins add editor integration, project templates, code completion, debugging and the Flutter Inspector. They DO NOT install the Flutter command-line SDK. Install Flutter separately in Step 06. The Flutter SDK includes the Dart SDK, so a second standalone Dart SDK is NOT required for these exercises.

PLUGIN + SDK VERIFICATION | Run AFTER Step 06 in a new terminal

```
flutter --version
dart --version
flutter doctor -v
```

Successful check: Android Studio shows Flutter AND Dart under Installed plugins, both commands print versions, and flutter doctor recognizes the Android toolchain. If the plugin is installed but flutter is not a recognized command, fix the Flutter SDK PATH rather than reinstalling the plugin.

STEP 06 | Install the Flutter SDK and update PATH

Download the stable Flutter SDK from https://docs.flutter.dev/install/manual. Extract the SDK to a user-owned directory (for example C:\src\flutter), rather than Program Files. Add its bin directory to your user PATH and restart terminals. Installing only the Android Studio plugin does NOT install the Flutter SDK.

```
C:\src\flutter
C:\src\flutter\bin    (add this directory to your user PATH)
```

STEP 07 | Validate the toolchain and accept Android licenses

Run Flutter Doctor and accept Android SDK licenses. Confirm that Android toolchain and Android Studio are detected; the connected phone will be checked in the next section. If needed, point Flutter to the exact Android SDK location shown in SDK Manager.

```
flutter doctor -v
flutter doctor --android-licenses
# Only if Flutter cannot locate your actual installed Android SDK:
flutter config --android-sdk "ACTUAL_SDK_PATH"
```

STEP 08 | Create a Flutter project

In Android Studio select New Flutter Project > Flutter, choose the installed Flutter SDK, name your project and select Android. You can also create a project from a terminal. Use a reverse-domain organization ID for real projects.

Guided walkthrough: create your first app with Android Studio

WIZARD 1 | Launch the correct project wizard

From the Welcome screen choose New Flutter Project. If another project is open, select File > New > New Flutter Project. On IDE versions that first show a Generators list, select Flutter; choose an Application rather than a Plugin or Module.

WIZARD 2 | Select the Flutter SDK DIRECTORY

When asked for Flutter SDK path, browse to the folder containing bin/, packages/ and other Flutter SDK files (for example C:\src\flutter). Do not select C:\src\flutter\bin and do not select the Android SDK. The IDE may display a Next button after it locates a valid SDK.

WIZARD 3 | Enter project metadata carefully

Use the following worked example. Some Android Studio versions show slightly different field names. A project name uses lowercase letters, numbers and underscores; it is NOT the visible app name on the phone.

Wizard field	Example / explanation
Project type	Flutter Application / Application, not Module or Plugin.
Project name	mad_wizard_demo (valid Dart package name; avoid spaces and hyphens).
Description	CSC303 practice: first Android Flutter application.
Project location	Select a writable folder under your own account; ensure the final project directory is mad_wizard_demo, not a duplicate nested folder.
Organization	edu.cui.sahiwal (classroom example used in this manual; use your own unique reverse-domain identifier for publishing).
Platforms	Select Android. iOS, web and desktop are unnecessary for these phone-focused labs.
Android language	Keep Kotlin if Android Studio offers this option.

WIZARD 4 | Finish and let Android Studio initialize the project

Click Create or Finish. Wait while files are generated and Dart/Flutter packages are resolved. Android Studio may also index code and download Gradle dependencies on first run; this needs an internet connection.

WIZARD 5 | Open the actual Flutter project root

In the Project tool window, change the tree-view selector to Project or Project Files if you only see the Android view. Verify pubspec.yaml, lib/main.dart, android/ and test/ appear together under mad_wizard_demo. A directory containing only android/ is not the Flutter project root.

WIZARD 6 | Select and run on your physical Android phone

Connect the phone and approve its USB debugging authorization (Part B below). Select the phone in Android Studio device dropdown. Open lib/main.dart and click the green Run triangle; the starter counter screen should launch on the PHONE. You can alternatively run flutter run from the project-root Terminal.

WIZARD 7 | Confirm your successful first build

Tap the starter counter + button and confirm its displayed count changes. Edit a displayed label, save, and trigger Hot Reload while the app runs. Record the connected device name, one screen capture and whether the toolchain passes flutter doctor.

EQUIVALENT COMMAND-LINE METHOD | Use a DIFFERENT name if the wizard app already exists

```
flutter create --platforms=android --org edu.cui.sahiwal mad_cli_demo
cd mad_cli_demo
flutter pub get
flutter devices
flutter run
```

After the code: flutter create generates the project with an Android platform target; cd enters the folder containing pubspec.yaml; flutter pub get resolves the dependencies; flutter devices lists available targets; flutter run builds and installs the app on your selected connected phone. If several devices are listed, the tool prompts you to choose one, or you can use flutter run -d followed by the exact ID it shows.

Important distinction: the wizard and terminal are ALTERNATIVE ways to create an app. Do NOT run flutter create into an already populated directory simply to repeat the wizard. Solved Task 1 later in this lab intentionally uses flutter create . inside the newly initialized csc303-mad Git repository; that is a separate, documented workflow.

First-run troubleshooting: identify the failing layer

Symptom	Likely cause and corrective action
New Flutter Project is missing	Enable/install both IDE plugins and restart; installing Flutter SDK alone does not add IDE actions.
Flutter SDK path rejected	Point to the SDK ROOT directory (the one containing bin/), not bin/ or the Android SDK.
flutter: command not found	Add Flutter SDK bin/ to user PATH; restart all terminals and Android Studio.

Symptom	Likely cause and corrective action
No physical phone in device list	Check USB debugging, phone authorization prompt and DATA cable; run adb devices and flutter devices.
adb: unauthorized	Unlock phone, approve the host RSA fingerprint, then retry; revoke authorizations and reconnect if needed.
Android licenses not accepted	Run flutter doctor --android-licenses, accept all agreements, rerun flutter doctor -v.
Gradle or SDK download fails	Check stable network, Android SDK installation, free disk space and Gradle errors in the Build tool window.
Only android/ appears in Project pane	Switch IDE Project pane from Android to Project / Project Files to reveal lib/, test/ and pubspec.yaml.

Student checkpoint: show the instructor (1) both installed plugins, (2) the Flutter SDK root, (3) an Android SDK platform and Platform-Tools, (4) a listed and authorized phone, and (5) an app running on the phone. Official reference: https://docs.flutter.dev/reference/create-new-app.

FINAL CHECK | Run these commands individually in Git Bash / terminal

```
git --version
ssh -V
flutter --version
flutter doctor -v
flutter doctor --android-licenses
flutter devices
```

B. Configure a physical Android phone — no emulator required

STEP 01 | Enable Android developer options

On the Android phone, open Settings > About phone, tap Build number approximately seven times (the menu can vary by manufacturer), then open Developer options and enable USB debugging. Disable debugging when it is no longer needed.

STEP 02 | Connect and authorize the computer

Connect with a USB DATA cable, unlock the phone, select File Transfer (MTP) if prompted, and approve the displayed RSA fingerprint only if it belongs to your computer.

STEP 03 | Install an OEM driver if Windows cannot detect the phone

If Windows Device Manager reports an unknown USB phone, install the manufacturer-supplied OEM USB driver. Android SDK Platform-Tools supplies adb, but the Windows hardware driver is a separate component on some devices.

STEP 04 | Confirm device authorization with ADB

Run adb devices and check the status. The phone should show device, NOT unauthorized. Then run flutter devices and select the phone in Android Studio or use flutter run with its device ID.

STEP 05 | Deploy, hot reload and build the APK

The first build may download Gradle and Android libraries, so remain connected. Press r in the terminal for hot reload while running. When ready, build a release APK using flutter build apk --release; the output is placed in build/app/outputs/flutter-apk/.

DEVICE CHECK / RUN / APK

```
adb devices
flutter devices
flutter doctor -v
# Use the ID printed by flutter devices when more than one target is attached.
flutter run -d ANDROID_DEVICE_ID
# Build an installable release package after your app works.
flutter build apk --release
```

FIELD NOTE ● If adb is not found, use Android Studio Terminal after SDK Platform-Tools installation or add the actual SDK platform-tools directory to PATH. For a device marked unauthorized, unplug, revoke USB debugging authorizations on the phone, reconnect and approve the new prompt.

C. Git essentials: explain the workflow before pushing

Concept	Technical meaning / recommended command
Working tree	Files currently edited; inspect with git status and git diff.
Index (staging)	Proposed next snapshot; git add <path>, git diff --cached.
Commit	Local immutable history snapshot; git commit -m "...".
Branch	Named reference to a line of commits; git switch -c feat/task-name.
Remote	Reference to hosted repository; git remote -v, git fetch origin.
Push / pull	Push publishes local commits; pull fetches + integrates remote changes.
Merge / rebase	Integrate change history; rebase rewrites local commit IDs—never rewrite shared history casually.
Ignore rules	Git .gitignore prevents generated files, local secrets and build artifacts from being staged.

D. Set up GitHub authentication using ssh-keygen

GitHub SSH authentication uses two files: a private key, which MUST remain on your computer, and a .pub public key, which you register with GitHub. Use a passphrase to protect the private key. Follow the steps in order, and never email, upload or commit your private key.

STEP 01 | Set your Git identity

This identity appears in commit metadata; use the email associated with your GitHub account.

```
git config --global user.name "Your Full Name"
git config --global user.email "YOUR_GITHUB_EMAIL@example.com"
git config --global init.defaultBranch main
```

STEP 02 | Check for an existing key, then create an ED25519 key

Use an existing secure key if appropriate. Otherwise run ssh-keygen; accept the default path unless it would overwrite an existing key, and set a strong passphrase.

```
ls -al ~/.ssh
ssh-keygen -t ed25519 -C "YOUR_GITHUB_EMAIL@example.com"
```

STEP 03 | Start ssh-agent and load your PRIVATE key locally

The agent remembers the unlocked key during your session. It does not upload the key to GitHub.

```
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519
```

STEP 04 | Display your PUBLIC key for GitHub

Copy the ENTIRE line printed below (normally starts with ssh-ed25519). On GitHub: avatar > Settings > SSH and GPG keys > New SSH key > Authentication key. Paste the public key, set a descriptive device title and save.

```
cat ~/.ssh/id_ed25519.pub
```

STEP 05 | Test the GitHub SSH connection

On the first connection, verify the GitHub host fingerprint against its official documentation before accepting it. A greeting containing your GitHub username means SSH authentication succeeded. GitHub offers no shell, so the test can end with exit code 1.

```
ssh -T git@github.com
```

SECURITY CHECK | Private key stays local; upload ONLY the .pub key

```
# NEVER upload ~/.ssh/id_ed25519 to GitHub.
# Upload ONLY ~/.ssh/id_ed25519.pub as an Authentication key.
```

If ssh -T reports permission denied (publickey), check whether the correct key is loaded with ssh-add -l, whether the .pub file was registered on the correct GitHub account, and whether your Git remote begins with git@github.com:. Never solve this by sharing private keys.

ALTERNATIVE WINDOWS POWERSHELL — run first two lines with administrator privileges if necessary

```
Get-Service -Name ssh-agent | Set-Service -StartupType Manual
Start-Service ssh-agent
ssh-add "$HOME/.ssh/id_ed25519"
Get-Content "$HOME/.ssh/id_ed25519.pub"
```

A passphrase protects the private key at rest. The agent remembers unlocked key credentials for the current session. If Git Bash and Windows SSH-agent conflict, configure Git to use Windows OpenSSH with git config --global core.sshCommand "C:/Windows/System32/OpenSSH/ssh.exe".

Solved task 1 | Initialize, branch, commit and push a repository

Problem and objective. Create an empty GitHub repository named csc303-mad (do NOT initialize it with a README on GitHub). In Git Bash, create the matching LOCAL folder and generate a real Flutter application in its root before your first commit. Replace YOUR_USERNAME with your GitHub username. A private repository is acceptable if the instructor has reviewer access.

GIT BASH — project setup

```
mkdir csc303-mad
cd csc303-mad
git init -b main
git config --local user.name "Your Full Name"
# Create an Android Flutter app IN the Git repository, not as a sibling.
# A Dart package name cannot contain a hyphen, hence --project-name.
flutter create --platforms=android --project-name csc303_mad --org edu.cui.sahiwal .
printf "# CSC303 Flutter Laboratory\n\nFlutter Android coursework.\n" > README.md
# APPEND to Flutter's generated .gitignore; do NOT overwrite it.
printf "\n# CSC303 local-only secrets\n.env\n*.jks\n*.keystore\n" >> .gitignore
git status
git add .
git diff --cached --stat
git commit -m "chore: initialize Flutter project"
git remote add origin git@github.com:YOUR_USERNAME/csc303-mad.git
git push -u origin main
# Make a reviewable docs branch instead of editing main directly.
git switch -c docs/lab01-readme
printf "\n## Lab 01\nGit and Flutter setup verified.\n" >> README.md
git add README.md
git commit -m "docs: document Lab 01 setup"
git push -u origin docs/lab01-readme
```

How the solution works. git init builds the repository, but flutter create is what makes it a runnable Android app. The folder is named csc303-mad for GitHub while --project-name sets the legal Dart package name csc303_mad. Flutter generates pubspec.yaml, lib/, android/ and the initial test/ directory; its generated .gitignore must be preserved. Staging and committing snapshot the code; the SSH remote connects that snapshot to GitHub. The documentation branch allows review without direct edits to main.

Expected output. The repository root contains pubspec.yaml, lib/main.dart, android/ and README.md. GitHub shows main plus docs/lab01-readme; open a pull request from the documentation branch into main. The starter Flutter app is a genuine buildable project.

Check your work. Run git log --oneline --decorate -3, git status and git remote -v; the working tree should be clean.

Solved task 2 | Create, build and install the first Flutter app on the phone

Problem and objective. Use the SAME csc303-mad Flutter repository from Solved Task 1. Return to main, create a separate feature branch and replace lib/main.dart with the complete counter app below. Replace the generated starter test with the matching test below. Deploy to the physical phone, run tests and open a pull request for the feature.

GIT BASH / TERMINAL — from the csc303-mad repository root

```
# Begin IN the csc303-mad folder made in Task 1.
git switch main
```

```
git pull --ff-only origin main
git switch -c feat/first-phone-counter
flutter pub get
flutter devices
```

lib/main.dart — complete copy-paste file

```
import 'package:flutter/material.dart';

void main() => runApp(const FirstPhoneApp());

class FirstPhoneApp extends StatelessWidget {
  const FirstPhoneApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'My First Android App',
      theme: ThemeData(colorSchemeSeed: Colors.teal, useMaterial3: true),
      home: const FirstPhonePage(),
    );
  }
}

class FirstPhonePage extends StatefulWidget {
  const FirstPhonePage({super.key});
  @override
  State<FirstPhonePage> createState() => _FirstPhonePageState();
}

class _FirstPhonePageState extends State<FirstPhonePage> {
  int taps = 0;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('CSC303 • My Android phone')),
      body: Center(
        child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            const Icon(Icons.phone_android, size: 70),
            const SizedBox(height: 16),
            Text('Button tapped $taps times',
                style: Theme.of(context).textTheme.headlineSmall),
            const SizedBox(height: 12),
            FilledButton(
              onPressed: () => setState(() => taps++),
              child: const Text('Tap on my phone'),
            ),
          ],
        ),
      ),
    );
  }
}
```

test/widget_test.dart — replace the generated starter test

```
import 'package:flutter_test/flutter_test.dart';
import 'package:csc303_mad/main.dart';

void main() {
  testWidgets('counter increments when button is tapped', (tester) async {
    await tester.pumpWidget(const FirstPhoneApp());
    expect(find.text('Button tapped 0 times'), findsOneWidget);
    await tester.tap(find.text('Tap on my phone'));
    await tester.pump();
    expect(find.text('Button tapped 1 times'), findsOneWidget);
  });
}
```

TERMINAL — run from csc303-mad after saving BOTH Dart files

```
dart format lib test
flutter analyze
flutter test
flutter run
# Test on your phone; press q to quit flutter run before continuing.
flutter build apk --debug
# Commit the app AND the corrected widget test on the feature branch.
git add lib/main.dart test/widget_test.dart
git commit -m "feat: add first Android counter and widget test"
git push -u origin feat/first-phone-counter
# Open a GitHub PR from feat/first-phone-counter to main.
```

How the solution works. MaterialApp configures the app, Scaffold supplies the page structure, and StatefulWidget holds the mutable tap count. setState schedules a redraw after each press. The widget test checks the counter before and after tapping; it replaces the generated test that still expects the old MyApp class. Git records the app and test together on a feature branch, enabling review before merge.

Expected output. The physical phone shows an app bar, phone icon, counter and working button. flutter analyze and flutter test complete successfully. The feature branch is visible on GitHub and can be opened as a reviewed pull request.

Check your work. Run flutter analyze and flutter test; tap five times on your Android phone. Check git status, ensure the new test is committed, and demonstrate a pull request with both app code and test in its Files changed tab.

Assignment 1 — mandatory complete contribution guide (5 marks)

Build a shared CSC303 Flutter repository with a protected main branch and use this workflow for every later lab. The students must commit README.md, CONTRIBUTING.md and the pull request template shown below. The following guide is both the project policy and a ready-to-copy contribution file.

README.md explains how to run the project; CONTRIBUTING.md defines the team rules; .github/PULL_REQUEST_TEMPLATE.md ensures every contribution includes reproducible checks and screenshots. Create each file at the exact location written above its example.

CONTRIBUTING.md — save in repository root (replace values where indicated)

```
# Contributing to CSC303 Flutter Project

## 1. Scope
Contribute Flutter/Dart, Android configuration, tests and docs only.
Discuss substantial features in a GitHub Issue before starting.
Never commit passwords, SSH private keys, service accounts or keystores.

## 2. Local setup
1. Fork the repository if you do not have write access.
2. Clone via SSH: git clone git@github.com:YOUR_USERNAME/csc303-mad.git
3. cd csc303-mad
4. flutter doctor -v; flutter pub get; flutter run

## 3. Branches and commit messages
Create a branch from the latest main:
  git switch main
  git pull --ff-only origin main
  git switch -c feat/short-description
Use feat/, fix/, docs/, test/, refactor/ or chore/ prefixes.
Commit small units with informative messages such as:
  feat: add bookmark persistence
  fix: handle failed API requests
  test: cover empty cart state

## 4. Quality gates
Run dart format ., flutter analyze, and flutter test.
Build on a real Android phone for UI-facing changes.
Write a test for new business logic where practical.
Update README and screenshots when behavior changes.

## 5. Open a pull request
Push with git push -u origin YOUR_BRANCH_NAME.
Open a PR to upstream main. Link an issue using Closes #ISSUE_NUMBER.
```

```
Explain purpose, approach, manual tests and screenshots.
Request review from at least one team member.
Respond to review by pushing new commits to the same branch.
Do not merge failing checks. Use squash merge if the maintainer allows.

## 6. Conflicts and synchronization
With a fork, add upstream once:
  git remote add upstream git@github.com:COURSE_OWNER/csc303-mad.git
To synchronize:
  git fetch upstream
  git switch main
  git merge --ff-only upstream/main
Resolve conflicts on your feature branch, test again and push.
Do not force-push main; ask the maintainer before rewriting shared history.

## 7. Review checklist
[ ] Behavior matches the linked issue
[ ] Screenshots and README are updated
[ ] No secrets, generated builds or unrelated files
[ ] flutter analyze and flutter test pass
[ ] UI checked on an Android phone
[ ] Accessibility labels and empty/error states considered

## 8. Code of conduct
Provide specific and courteous technical feedback; credit contributors.
Plagiarism, copied submissions and fabricated results are prohibited.
The maintainer may request revisions before merging.
```

.github/PULL_REQUEST_TEMPLATE.md — create .github folder first

```
## Summary
Describe what changed and why.

## Related issue
Closes #ISSUE_NUMBER

## Validation
- [ ] dart format .
- [ ] flutter analyze
- [ ] flutter test
- [ ] Tested on a physical Android phone

## Screenshots / screen recording
Before / after:

## Risks and rollback
State network, database migration or compatibility risks.

## Reviewer notes
Mention difficult decisions, edge cases and files to inspect.
```

Contribution workflow: maintainer vs student

Repository maintainer. Create issues with clear acceptance criteria, label feature/bug/docs/test work, invite team members (or allow forks), protect main through branch rules when available, and merge only reviewed pull requests with checks.
Contributor. Claim an issue, branch from current main, implement one focused change, run dart format . / flutter analyze / flutter test, push the branch and open a PR with evidence.
Reviewer. Review correctness, usability, security and tests. Request changes when needed, and merge only after comments are resolved.

REVIEWED FEATURE BRANCH — execute from an existing Flutter repository

```
# Student with direct write access:
git switch main
git pull --ff-only origin main
git switch -c feat/contact-screen
# Edit your files, then run checks:
dart format .
flutter analyze
flutter test
```

```
git status
git add lib test README.md
git commit -m "feat: create contact screen"
git push -u origin feat/contact-screen
# Open a Pull Request on GitHub into main and request review.

# Optional: after merge, clean up the local feature branch:
git switch main
git pull --ff-only origin main
git branch -d feat/contact-screen
```

Conflict resolution exercise: two students change the same title in lib/main.dart. Create a second branch, make a different edit, merge, inspect <<<<<<< / ======= / >>>>>>> conflict markers, choose a correct single Dart expression, run flutter analyze, stage the resolved file and commit the merge. Do not push unresolved markers.

Scenario-based practice — unsolved

Scenario 1. Campus team repository. A three-person team must add a timetable feature and a UI bug fix without breaking main. Decide issue ownership, branch names, commit granularity, reviewer order and the evidence required for each PR. Deliver two issue links and two reviewed PRs.

Scenario 2. No device appears. A classmate installed Android Studio and Flutter but flutter devices reports no Android phone, while adb devices says unauthorized. Diagnose the exact permission, driver and toolchain steps; record commands/output and explain why an emulator image is not required.

Technical viva — justify your reasoning

1. Why is a commit not the same as a push?
2. What security property does a passphrase add to an SSH private key?
3. Why must only id_ed25519.pub be added to GitHub?
4. How does Android SDK Platform-Tools differ from Build-Tools?
5. Why does GitHub ssh -T sometimes exit with status 1 even after successful authentication?
6. What evidence tells you a USB connection is blocked at the ADB authorization stage?
7. How do a fork, remote origin and remote upstream differ?
8. Why should a team avoid force-pushing its protected main branch?

Submission and self-check

Submit screenshots of flutter doctor -v (hide personal paths if desired), adb devices, phone app, SSH authentication greeting, two commits and a reviewed PR; include CONTRIBUTING.md and the PR template. Never include your private key.

CSC303 / NOVEMBER 2026 REVISION

LAB 01 EXTENSION | Contribution workflow in the portfolio

The Lab 01 csc303-mad example continues to teach Git through its own starter project. From Lab 02 onward use the portfolio structure on the preceding page, or an instructor-provided course organization repository. Do not paste Flutter apps into the same project root as one another.

Repository owner versus contributor

Owner / maintainer	Student contributor
Creates portfolio, invites teammates; enables available branch rules and review settings	Clones repository; claims an issue; creates a feature branch
Reviews PRs and confirms required checks	Runs format, analyze, relevant tests and real-phone demonstration
Merges accepted PRs and manages release tag	Commits focused code; opens PR; responds to reviewer feedback

Repository protection controls depend on repository ownership and GitHub plan/settings. If protection cannot be configured in a particular classroom repository, require peer-reviewed pull requests by assessment policy and report the limitation instead of claiming protection is enabled.

Example next-lab contribution

```
# From csc303-portfolio/ after its main branch is pushed:
git switch main
git pull --ff-only origin main
git switch -c feat/lab02-dart
cd labs
flutter create --platforms=android mad_lab02
cd mad_lab02
# Edit source; run format, analyze and current tests.
dart format lib test
flutter analyze
cd ../..
git add labs/mad_lab02
git commit -m "feat: submit Lab 02 Dart app"
git push -u origin feat/lab02-dart
# Open PR; request a classmate review.
```

Acceptance evidence

- Show two meaningful commits, one successful pull request review, and the relevant Flutter project files in the PR diff.
- Show a real device recognized by ADB and a Flutter application launched on that device.
- Explain the role of origin/upstream; demonstrate that the private SSH key never leaves the device.
- Document whether the repository uses enforced branch protection or a classroom review-only procedure.